SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero - Junio 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de Diseño de Software

Grupo:
03:00 - 04:00 pm

Actividad:
Actividad de evidencia 1_U2 Abstract Factory

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Mendez Adame Noel Andre 2121193

NOMBRE DEL DOCENTE:

Maribel Guerrero Luis

LUGAR Y FECHA:

Tijuana, Baja California, México a 4 de Marzo del 2026

Abstract Factory

El patrón de diseño creacional Abstract Factory es un patrón que proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.

Su propósito es garantizar que los objetos creados sean compatibles entre sí, siendo útil cuando el sistema debe trabajar con distintas variantes de productos

Este patrón cumple con el principio de responsabilidad única (creación aislada) y el principio de abierto/cerrado (se pueden introducir nuevas familias sin romper el código existente).

Código implementado:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Abstract_Factory
{
    public abstract class  MaterialFactory
    {
        public abstract Guia CrearGuia();

        public abstract Examen CrearExamen();
    }

    public abstract class Guia
    {
        public abstract void Mostrar();
    }

    public abstract class Examen
    {
        public abstract void Aplicar();
    }
```

```
public class GuiaImpresa : Guia
{
    public override void Mostrar()
    {
        Console.WriteLine("Mostrando la guía impresa.");
    }
}

public class ExamenEnPapel : Examen
{
    public override void Aplicar()
    {
        Console.WriteLine("Se aplica examen en papel.");
    }
}

public class GuiaPDF : Guia
{
    public override void Mostrar()
    {
        Console.WriteLine("Mostrando la guía PDF.");
    }
}

public class ExamenEnLinea : Examen
{
    public override void Aplicar()
    {
        Console.WriteLine("Se aplica examen en línea.");
    }
}

public class GuiaHibrida : Guia
{
    public override void Mostrar()
    {
        Console.WriteLine("Mostrando la guía Semipresencial.");
    }
}
```

```
public class ExamenMixto : Examen
{
    public override void Aplicar()
    {
        Console.WriteLine("Se aplica examen mixto");
    }
}

public class MaterialPresencialFactory : MaterialFactory
{
    public override Guia CrearGuia()
    {
        return new GuiaImpresa();
    }
    public override Examen CrearExamen()
    {
        return new ExamenEnPapel();
    }
}


public class MaterialVirtualFactory : MaterialFactory
{
    public override Guia CrearGuia()
    {
        return new GuiaPDF();
    }
    public override Examen CrearExamen()
    {
        return new ExamenEnLinea();
    }
}

public class MaterialHibridoFactory : MaterialFactory
{
    public override Guia CrearGuia()
    {
        return new GuiaHibrida();
    }
    public override Examen CrearExamen()
    {
```

```
            return new ExamenMixto();
        }
    }


    internal class Program
    {
        static void Main(string[] args)
        {
            MaterialFactory fabrica;
            fabrica = new MaterialPresencialFactory();

            Guia guia = fabrica.CrearGuia();
            Examen examen = fabrica.CrearExamen();

            guia.Mostrar();
            examen.Aplicar();

            Console.WriteLine();
            fabrica = new MaterialVirtualFactory();

            guia = fabrica.CrearGuia();
            examen = fabrica.CrearExamen();

            guia.Mostrar();
            examen.Aplicar();
            Console.WriteLine();

            fabrica = new MaterialHibridoFactory();

            guia = fabrica.CrearGuia();
            examen = fabrica.CrearExamen();

            guia.Mostrar();
            examen.Aplicar();

            Console.ReadKey();

        }
    }
}
```

Conclusión de este patrón y la implementación realizada.

En conclusión, el patrón de diseño Abstract Factory facilita la organización y escalabilidad del sistema porque permite crear familias de objetos que estén relacionados sin depender de los demás que ya estén implementados.

En este caso se manejó todo de la misma manera siendo guia y examen, eso no se modificó, pero si se crearon familias de estos objetos haciendo variantes como exámenes presencial, en línea y mixto, al igual que guías presenciales, en línea y mixtos, donde la creación de más de estos no afecta a los demás ni a la principal que es examenes y guias. Se mantiene flexible esto para poder agregar más así como agregamos el híbrido, sin afectar al resto del sistema.